

Top 10 web hacking techniques of 2025



James Kettle
Director of Research
[@albinowax](#)



Published: Thursday, 5 February 2026 at 15:28 UTC

Updated: Friday, 6 February 2026 at 08:07 UTC



Welcome to the Top 10 Web Hacking Techniques of 2025, the [19th edition](#) of our annual community-powered effort to identify the most innovative must-read web security research published in the last year. This post is the culmination of a three-step collaboration with the security community. Over the last month:

- The community [submitted nominations](#) for the top research from 2025
- The community voted on those nominations to build a shortlist of the top 15
- An expert panel voted on the shortlist to select and order the 10 finalists

We're hoping to wrap up with an in-person award ceremony with physical prizes at a DEF CON village - stay tuned for further details on this.

This year, the community nominated 63 pieces of research as contenders. This is significantly fewer than the crazy 121 submissions last time, possibly because we collectively got distracted by AI. That said, it's back in line with historical nomination numbers from 2022 and 2023.

I was honoured to see the fifteen finalists from the community vote included my own talk HTTP/1.1 Must Die: the desync endgame, but as usual I've excluded it from the final top ten.

An expert panel consisting of [Nicolas Grégoire](#), [Soroush Dalili](#), [STÖK](#), [Fabian \(LiveOverflow\)](#), and [myself](#) has reviewed the finalists and we're thrilled to bring you the top ten web hacking techniques of 2025!

10 - Parser Differentials: When Interpretation Becomes a Vulnerability

In tenth place, we've got [Parser Differentials: When Interpretation Becomes a Vulnerability](#) by [@joernchen](#), featuring case-studies affecting a broad range of languages, frameworks and technologies. There's sadly no accompanying whitepaper but this presentation is an excellent starting point for someone looking for ideas to get started with their own research.

9 - Playing with HTTP/2 CONNECT

HTTP/2 has been around for a while now but still rewards researchers who aren't scared of RFC-diving and custom tool development. Whenever a new protocol emerges you'll find old flaws resurfacing in fresh code, and [Playing with HTTP/2 CONNECT](#) illustrates this succinctly, with internal port-scan tooling. As support for HTTP/2 CONNECT spreads, this research from [@flomb](#) is another great candidate to build on.

8 - XSS-Leak: Leaking Cross-Origin Redirects

Don't be fooled by the name - [XSS-Leak: Leaking Cross-Origin Redirects](#) by [Salvatore Abello](#) has nothing to do with [XSS](#). This beautiful attack uses Chrome's connection-pool prioritisation algorithm as an oracle to leak redirect hostnames cross-domain. Even if Chrome patches their algorithm, this post will remain valuable as an inspiration for future xs-leaks.

7 - Next.js, cache, and chains: the stale elixir

While standalone [web cache poisoning](#) is well-understood, internal cache poisoning remains an overlooked and distinctly scary variant. The moment I saw [Next.js, cache, and chains: the stale elixir](#) back in January last year, I knew it was destined for the top ten. In this writeup of a critical vulnerability in the heart of next.js, [Rachid Allam](#) shows how to use source-code analysis to piece together masterful attacks and naturally leaves us wondering what surprises are lurking in other popular frameworks.

6 - Cross-Site ETag Length Leak

The second XS-Leak to land in this year's top ten, [Cross-Site ETag Length Leak](#) was first discovered as an unintended solution to a CTF. [Takeshi Kaneko](#) crafts an elegant chain of multiple edge-cases to leak the response-size cross-domain. It takes the edge over the origin-leak technique due to being slightly more versatile - and harder to patch.

5 - SOAPwn: Pwning .NET Framework Apps Through HTTP Client Proxies And WSDL

[SOAPwn](#) starts with a single flaw in `HttpWebClientProtocol` that Microsoft refused to fix. [Piotr Bazydło](#) then gradually develops this into a powerful exploitation sink enabling RCE on a bunch of products. Don't be put off by the 93-page whitepaper - it's surprisingly easy to read.

4 - Lost in Translation: Exploiting Unicode Normalization

Unicode normalization attacks have lurked on the edge of testing methodologies for years, periodically grabbing the limelight before fading into the background. In [Lost in Translation](#), [Ryan & Isabella Barnett](#) tackle this vast research topic, combining diverse exploit samples with updates to third-party tools including [ActiveScan++](#). Ryan's unique vantage point at a major WAF vendor, seeing what attacks actually get used in the wild, makes this the highly practical talk that Unicode deserves.

3 - Novel SSRF Technique Involving HTTP Redirect Loops

"But why did it work?" This [technique for making blind SSRF visible from @shubs](#) is beautiful, simple and powerful. The detailed writeup of the discovery story provides a rare glimpse into the messy truth behind great research findings. There's some powerful takeaways here but I don't want to spoil them - read it closely, and contemplate. In the words of panelist Soroush, "that's magic".

2 - ORM Leaking More Than You Joined For

Like XS-leaks? ORM leaks are their chunky server-side cousin. [ORM Leaking More Than You Joined For](#) evolves ORM leaks from a niche, framework-specific vulnerability into a generic methodology for exploiting search and filtering capabilities. As [SQL injection](#) fades into the background, creative ways to dump the database are always welcome. A well earned #2 for this research from [Alex Brown](#).

1 - Successful Errors: New Code Injection and SSTI Techniques

Successful Errors: New Code Injection and SSTI Techniques introduces new error-based techniques for exploiting blind [server-side template injection](#). This superb analysis also includes novel polyglot-based detection techniques to comprehensively expose this attack class. By adapting old-school techniques associated with SQL injection, and integrating these into a powerful open-source toolkit, [Vladislav Korchagin](#) might just have ushered in a new era of server-side template injection. Congrats on a hard-earned win!

Conclusion

2025 saw the rise of side-channels as a core exploitation primitive. It'll be interesting to see if this trend continues for 2026 - or vibe-coding going mainstream takes us back to the bad old days.

[As always, with 63 nominations many great writeups didn't make the final fifteen, let alone the top ten!](#)
[Here's a tiny sample of some of the delights awaiting you in the full nomination list.](#)

- Novel desync techniques involving malformed chunks
- Multiple techniques to stall browser redirects, enabling further exploit chains
- New SAML exploitation techniques enabling complete authentication bypass

Also, if you spotted some exceptional research from 2025 that never got nominated, chuck me an email and I'll add it to the list.

Part of what lands an entry in the top 10 is its expected longevity, so it's well worth getting caught up with [the top ten archive](#) too. If you're interested in getting a preview of what might win from 2026, you can [subscribe to our RSS](#), join [r/websecurityresearch](#), hop on [our Discord](#), or follow us on social. If you're interested in doing this kind of research yourself, I've shared a few lessons I've learned over the years in [Hunting Evasive Vulnerabilities](#), [How to choose a security research topic](#), and [So you want to be a web security researcher?](#)

Massive thanks to the panel for contributing their time and expertise to curating the final result, and thanks also to everyone who took part! Without your nominations, votes, and most-importantly research, this wouldn't be possible.

Till next time!

Top 10 Hacking Techniques

top-10-techniques

← [Back to all articles](#)

Related Research



Burp Suite

Web vulnerability scanner
Burp Suite Editions
Release Notes

Vulnerabilities

Cross-site scripting (XSS)
SQL injection
Cross-site request forgery
XML external entity injection
Directory traversal
Server-side request forgery

Customers

Organizations
Testers
Developers

Company

About
Careers
Contact
Legal
Privacy Notice
Modern Slavery Statement

Insights

Web Security Academy
Blog
Research
Engineering

 **PortSwigger**

 [Follow us](#)

© 2026 PortSwigger Ltd.

